

auto_sms

동구청 민원문자 관리 시스템

01 개요

02 아키텍처

03 데이터 모델

04 인증 · 보안

05 API 레퍼런스

06 스레드 묶기 로직

07 감시 데몬 · phone_link.py

08 업무외 자동발송

09 프론트엔드 구조

10 운영 노트

휴대폰과 연결 앱 자동화부터 스레드 로직까지

Flask 기반 민원 문자 대시보드의 아키텍처, 데이터 모델, API, 휴대폰과 연결 (Phone Link) 앱을 pywinauto로 조작하는 감시·발송 데몬, 스레드 뭉치·업무외 자동발송 로직을 코드 기준으로 정리했습니다.

스택 Flask 3 · SQLite(WAL) · Jinja2 · Vanilla JS · pywinauto(Windows 전용)

코드 규모 app.py 856줄 · phone_link.py 926줄 · watch_daemon.py 275줄 · dashboard.js 1009줄

01 · 개요

휴대폰과 연결(Phone Link, 구 Your Phone) 앱을 통해 오가는 민원 문자를 자동으로 감지해 저장하고, 상용문구로 원클릭 답신을 보내는 3인 규모 팀용 내부 도구입니다.

서버(`app.py`)는 `python app.py` 한 줄로 뜨는 단일 Flask 프로세스이고, 실제 문자 감시는 별도 프로세스인 `watch_daemon.py` 가 맡습니다. 둘 다 `phone_link.py` 를 통해 Windows의 "휴대폰과 연결" 앱 창을 pywinauto(UI Automation)로 직접 조작합니다 — 이 앱이 제공하는 공식 API가 없어서, 사람이 마우스/키보드로 하는 조작을 그대로 흉내 내는 방식입니다.

대시보드 서버 자체(메시지 목록, 상용문구 관리, 계정 관리)는 OS 무관하게 쓰지만, 실제 발송(`/api/send`)과 감시 데몬은 pywinauto가 Windows 전용(pywin32 의존)이라 그 기능만 Windows PC에서만 동작합니다 — `phone_link` 관련 import는 전부 함수 안에서 지연 import되어, 리눅스/개발 환경에서도 앱 자체는 문제없이 켜집니다.

구성 요소	실행 위치	역할
<code>app.py</code>	Windows PC (OS 무관하게 뜨긴 함)	Flask 대시보드 서버, SQLite 접근, 세션/권한, 발송 API
<code>watch_daemon.py</code>	Windows PC (app.py와 같은 PC)	휴대폰과 연결 앱 화면을 주기적으로 훑어 새 문자를 <code>app.py</code> 에 POST

구성 요소	실행 위치	역할
<code>phone_link.py</code>	-	위 둘이 공유하는 라이브러리. pywinauto로 휴대폰과 연결 창을 찾아 읽고/쓴다
SQLite (<code>auto_sms.db</code>)	로컬 파일	messages/templates/settings/users 4개 테이블. WAL 모드

02 · 아키텍처

브라우저가 다른 애플리케이션의 창을 직접 읽을 수 없는 건 `auto_sms`도 마찬가지입니다 — 그래서 "별도 프로세스가 그 창을 대신 읽어 HTTP로 서버에 올려준다"는, NDMS의 `situation_capture.py`와 같은 큰 구조를 씁니다.

수신 흐름

1

휴대폰이 문자 수신 → 블루투스/Wi-Fi로 PC의 휴대폰과 연결 앱에 동기화

↓

2

`watch_daemon.py` 가 5초 간격으로 `phone_link.watch_notifications()` 호출 — 알림 패널을 polling

↓

3

새 줄 발견 시 `POST /api/messages` (phone_number/body/msg_time)

↓

4

app.py가 dedup_key로 중복 확인 후 저장, 업무의 자동발송 여부 판단 후 별도 스레드로 디스패치

발송 흐름

1

대시보드 "발송" 버튼 → `POST /api/send` (또는 자동발송 트리거)



2

app.py가 `_send_lock` 으로 동시 발송 직렬화 후 `phone_link.send_message()` 호출



3

pywinauto가 휴대폰과 연결 창을 복원 → 대화 찾기/새로 만들기 → 타이핑 → 전송 버튼 클릭



4

DB에 발신 메시지 기록, 원래 최소화 상태였으면 창을 다시 최소화

단일 프로세스 + threaded=False인 이유

`app.run()` 이 기본값(`threaded=False`)으로 요청을 한 번에 하나씩만 처리합니다. 업무 외 자동발송(`_maybe_send_auto_reply`)처럼 몇 초 걸릴 수 있는 UI 자동화를 요청 스레드에서 그대로 기다리면, 그사이 들어오는 다른 요청(특히 `watch_daemon.py` 의 수신 문자 저장)이 전부 막혀버립니다. 그래서 `_dispatch_auto_reply()` 가 별도 데몬 스레드로 넘겨, 수신 문자 저장 응답은 항상 즉시 돌아가고 서버가 계속 다른 요청을 받을 수 있게 합니다.

참고

새로 뜨는 스레드에서 `pywinauto`(UI Automation/COM)를 처음 쓰려면 그 스레드 용으로 `pythoncom.CoInitializeEx()` 를 따로 호출해야 합니다 — 메인 스레드에서 이미 COM을 썼다고 다른 스레드까지 적용되지 않습니다.

`_dispatch_auto_reply()` 의 `run()` 내부가 이걸 처리하고, 실패하면(리눅스 등) 조용히 건너뛸니다.

동시 발송 직렬화

휴대폰과 연결 창은 하나뿐인데, "발송" 버튼(`/api/send`)과 업무 외 자동발송(별도 스레드)이 동시에 겹치면 같은 창을 두 흐름이 같이 건드려 엉뚱한 대화방에 타이핑되거나 문자가 섞일 수 있습니다. 모듈 전역 `_send_lock = threading.Lock()` 으로 실제 발송(`phone_link.send_message()` 호출)은 항상 한 번에 하나씩만 진행되게 막습니다.

디렉터리 구성

```
auto_sms/
├─ app.py                # Flask 앱, 라우트, 세션/CSRF, 자동발송 디스패치
├─ db.py                 # SQLite 연결/스키마/마이그레이션
├─ phone_link.py         # pywinauto로 휴대폰과 연결 앱 조작 (926줄)
├─ watch_daemon.py       # 상시 감시 데몬 (별도 프로세스)
├─ seed_demo_data.py     # 데모/스크린샷용 더미 데이터 삽입 스크립트
├─ start_server.bat / stop_server.bat
├─ templates/
│   └─ login.html
│   └─ dashboard.html    # 586줄, 단일 페이지 대시보드
├─ static/js/dashboard.js # 1009줄, 전체 화면 로직
├─ auto_sms.db           # SQLite (git 미포함)
└─ requirements.txt
```

03 · 데이터 모델

4개 테이블, 전부 `db.py` 의 `init_db()` 가 시작시 `CREATE TABLE IF NOT EXISTS` 로 생성합니다. WAL 모드 + `busy_timeout=5000` 으로 대시보드(여러 사람 동시 접속)와

watch_daemon.py (별도 프로세스)가 같은 DB 파일에 거의 동시에 쓰는 경합을 견뎌냅니다.

messages

컬럼	타입	비고
phone_number	TEXT	상대 번호. 폰 주소록 저장 이름으로 쓰는 경우 그 문자열이 그대로 들어갈 수 있음(전화번호 형식이 아닐 수 있음)
direction	TEXT	<code>in</code> (수신)/ <code>out</code> (발신) 체크 제약
dedup_key	TEXT UNIQUE	<code>phone_number msg_time body</code> . 위치가 같은 화면을 반복해서 훑어도 중복 저장 방지. 발신 기록은 NULL(SQLite는 NULL끼리 UNIQUE 충돌 안 봄)
status	TEXT	처리상태(미처리/답변완료(조치 예정)/증장기검토/처리기한 미도래/처리불가/처리완료/타기관이송) — 자유 텍스트 컬럼이지만 화면은 고정 목록만 씀
manual_input / receipt_no	TEXT	담당자가 목록에서 직접 입력하는 자유 텍스트(입력 경로/접수 번호)
thread_id	INTEGER	수동 병합되었거나 자동발송/발송으로 명시적으로 앵커링된 스레드 묶음 id. 기본 NULL — 없으면 <code>_build_threads()</code> 가 번호+시간순으로 추정해서 묶음 (06장)
auto_sent	INTEGER	업무외 자동발송으로 나간 답신이면 1 — 화면에 "자동" 배지

뒤 5개 컬럼(status~auto_sent)은 `_migrate_message_columns()` 가 `PRAGMA table_info` 로 존재 여부를 확인해 없으면 `ALTER TABLE` 로 추가합니다 — 이미 운영 중

인 DB에 코드만 업데이트해도 무중단으로 스키마가 따라잡히는 자체 마이그레이션 패턴입니다.

templates · settings · users

테이블	구조	용도
templates	id, title, body, sort_no	상용문구. sort_no로 드롭다운 표시 순서 지정
settings	key TEXT PK, value TEXT	범용 키-값 저장소 — Flask secret key, watcher heartbeat 시각, 업무외 자동발송 설정 (enabled/template_id/quiet_minutes) 전부 여기
users	id, username, password_hash, is_admin	werkzeug <code>generate_password_hash</code> 로 저장. 계정이 하나도 없으면 첫 로그인 화면이 관리자 계정 생성 폼으로 바뀐(04장)

참고

Flask 세션 서명키(`flask_secret_key`)를 코드에 박아두지 않고, 서버가 처음 뜰 때 `secrets.token_hex(32)` 로 한 번 생성해 `settings` 테이블(로컬 DB, git 미 포함)에 저장하고 계속 재사용합니다 — 재시작마다 값이 바뀌면 팀원 전체 로그인이 풀리는 걸 방지합니다.

04 · 인증 · 보안

세션 쿠키 + Origin/Referer 기반 CSRF 방어 + 로컬호스트 한정 예외 2개 (watch_daemon 전용). 규모(3명)에 맞춰 최소한으로 구성되어 있습니다.

첫 실행 부트스트랩

`/login` 라우트가 `SELECT COUNT(*) FROM users` 로 계정이 하나도 없는지 확인합니다 (`first_run`). 없으면 로그인 폼 대신 "아이디/비밀번호를 그 자리에서 정해 첫 관리자 계정을 만드세요" 폼으로 렌더링됩니다 — 기본 비밀번호를 코드에 미리 박아두지 않기 위한 설계입니다.

CSRF / Same-Origin 검사

@app.before_request 로 등록된 `_verify_same_origin()` 이 모든 POST에 대해 `Origin` (없으면 `Referer`) 이 이 서버 자신인지 확인하고, 아니면 403을 던집니다. `violation` 프로젝트의 같은 검사와 동일한 방식입니다.

로그인 없이 접근 가능한 예외 (`_LOCAL_MACHINE_ENDPOINTS`)

`watch_daemon.py` 는 브라우저가 아니라 `requests` 로 직접 호출하므로 로그인 세션 쿠키도, `Origin/Referer` 헤더도 없습니다. 그래서 `api_save_message` · `api_heartbeat` 두 엔드포인트만 로그인/CSRF 검사에서 빠지는데, 대신 `_is_localhost_request()` (`request.remote_addr` 가 `127.0.0.1:::1`) 로 "이 서버 자신에서 온 요청"인지 확인합니다 — `watch_daemon.py` 는 항상 서버와 같은 PC에서 돌기 때문에 이 조건으로 외부 접근을 차단할 수 있습니다.

검사	적용 대상	실패 시
<code>_verify_same_origin</code>	모든 POST (2개 예외 제외)	403
<code>_require_login</code>	로그인/정적파일 제외 전체	API: 401 JSON / 페이지: <code>/login</code> 리다이렉트
<code>_is_localhost_request</code>	<code>api_save_message</code> , <code>api_heartbeat</code> 만	위 두 검사(로그인/CSRF)를 그대로 적용받음

계정 관리

관리자만 `/api/users` (목록/생성), `/api/users/<id>/reset_password` 를 쓸 수 있습니다. 이메일 인증 등 본인확인 절차 없이 관리자가 팀원 비밀번호를 직접 재설정하는 방식으로, 3인 규모(관리자가 팀원을 서로 다 아는 사이)에 맞춘 의도적으로 단순한 설계입니다.

05 · API 레퍼런스

모든 `/api/*` 실패 응답은 `{"error": "..."} JSON`입니다.

메서드	경로	권한	설명
<code>GET</code> / <code>POST</code>	<code>/login</code>	-	첫 실행이면 관리자 계정 생성 폼, 아니면 일반 로그인
<code>POST</code>	<code>/logout</code>	로그인	세션 클리어

메서드	경로	권한	설명
GET	/api/users	admin	계정 목록
POST	/api/users	admin	계정 생성
POST	/api/users/<id>/reset_password	admin	비밀번호 강제 재설정
GET	/api/messages	로그인	스레드 목록 조회. phone / since / until / limit 쿼리. 응답은 _build_threads() 가 뮴은 {"threads": [...]}
POST	/api/messages	로컬호스트만	watch_daemon.py 전용. 새 수신 문자 저장 + 업무외 자동발송 트리거
PATCH	/api/messages/<id>/status	로그인	처리상태만 변경
PUT	/api/messages/<id>	로그인	입력/접수번호 동시 저장(항상 둘 다 같이 보냄)
POST	/api/messages/delete	로그인	선택한 스레드(+답신) 일괄 삭제
POST	/api/threads/merge	로그인	스레드 2개 이상을 하나의 thread_id로 병합
GET	/api/templates	로그인	상용문구 목록(sort_no 순)
POST	/api/templates	로그인	추가
PUT / DELETE	/api/templates/<id>	로그인	수정 / 삭제
POST	/api/heartbeat	로컬호스트만	watch_daemon.py 전용. 감시 생존 신호

메서드	경로	권한	설명
GET	/api/status	로그인	감시 생존 여부, 업무 시간 여부, 자동발송 on/off — 상단 배지가 켜
GET / PUT	/api/settings/auto_reply	로그인	업무외 자동발송 설정 조회/저장
POST	/api/send	로그인	실제 문자 발송 (Windows 전용, phone_link 지연 import)
POST	/api/shutdown	로그인	서버 프로세스 자체 종료

/api/messages GET의 기간 필터는 `since / until` (YYYY-MM-DD)만 받고, 예전 `limit=100` 단독 방식(몇 달 쌓이면 지난달 민원이 검색에도 안 잡히는 문제가 있었음)을 대체합니다. `limit`은 이제 한 번에 너무 많이 실어보내지 않기 위한 상한 역할만 합니다.

06 · 스레드 묶기 로직 (`_build_threads()`)

한 민원(수신 문자)에 답신(발신 문자)이 여러 개 이어질 수 있어서, 평평한 메시지 목록을 "민원 + 그 이후 답신들" 스레드 단위로 묶는 함수입니다. 이 앱에서 가장 까다로운 로직입니다.

우선순위: `thread_id` > 시간순 추정

`thread_id` 가 찍혀 있는 메시지는 그 값이 같은 것끼리 무조건 한 스레드로 묶입니다 — 체크박스 수동 병합된 경우(`/api/threads/merge`)뿐 아니라, "+ 문자발송"으로 보낸 답신도 어느 민원 밑 send-box에서 나갔는지(`complaint_id`)에 따라 명시적으로 묶입니다.

`thread_id` 없이 들어오는 발신 문자(업무외 자동발송의 옛 방식, `watch_daemon`이 감지한 옛 데이터 등)에 한해서만, "같은 번호로 그 민원보다 나중에 온 발신 문자는 답신"이라는 시간순 휴리스틱으로 보조적으로 묶습니다.

왜 시간순만으론 안 되는가

같은 번호로 서로 다른 민원이 따로 와 있을 수 있습니다. 번호만 보고 "그 민원보다 나중에 온 발신 문자는 다 답신"으로 넘겨짚으면, 오래된 민원에 대한 답신이 그 번호의 **최신** 민원 스레드로 잘못 붙어버리는 사고로 이어집니다(실제로 확인된 문제) — 그래서 발송(`/api/send`)과 자동발송(`_maybe_send_auto_reply`) 둘 다 `complaint_id` 를 명시적으로 받아 앵커링합니다.

자기 앵커링

어떤 민원이 아직 한 번도 병합/앵커된 적 없으면(`thread_id IS NULL`), 발송 시점에 자기 자신의 `id`를 `thread_id`로 삼아 스스로를 앵커로 만듭니다. 이렇게 해주면 발송이 진행되는 몇 초 사이 같은 번호로 다른 민원이 새로 들어와도(`_build_threads()` 의 "이 번호의 현재 열린 스레드" 포인터가 옮겨가도) 답신은 항상 자신을 유발한 민원 밑에 정확히 붙습니다. `/api/send` 와 `_maybe_send_auto_reply()` 양쪽에 동일한 패턴이 반복됩니다.

범위 한계

GET으로 넘어온 `limit` 범위 안에서만 묶으므로, 아주 오래된 민원의 답신이 지금 화면에 안 보이는 범위에 있으면 별도 스레드로 갈라져 보일 수 있습니다. "최근 메시지" 목록이 최근 활동을 보여주는 용도라 실용적으로 충분하다고 보고, 정확성을 위해 전체 이력을 매번 조회하지는 않습니다.

07 · 감시 데몬 · phone_link.py

926줄짜리 `phone_link.py` 가 이 프로젝트의 핵심입니다 — 공식 API가 없는 Windows 앱을 UI Automation만으로 안정적으로 읽고 쓰는 데 코드 대부분이 쓰였습니다.

창 찾기

창 제목이 아니라 프로세스명(`PhoneExperienceHost.exe`) 기준으로 먼저 찾습니다 — 제목 문자열 매칭은 우연히 같은 단어가 들어간 다른 창(메모장 등)을 잘못 잡을 위험이 있어서입니다. 최소화된 창도 찾을 수 있도록 `visible_only=False` 로 검색하는 `_top_window_any_state()` 를 별도로 둡니다 — `pywinauto uia` 백엔드의 기본 "visible" 판정이 `IsOffscreen` 기준이라, 최소화된 창은 기본 검색에서 아예 안 잡히기 때문입니다.

감시 방식 2가지

알림 패널 (기본,

`watch_notifications()`)

홈 화면 "알림" 패널

(`NotificationsListScrollHost`)을 훑음.

블루투스만으로도 채워지는 걸 확인해, Wi-Fi가 막힌 행정망 PC에서도 동작. 대신 미리 보기라 원문이 길면 잘릴 수 있음.

대화 목록 (`--source messages`)

`CVSListView` 를 훑음. 본문 동기화에 Wi-Fi가 필요하지만 원문이 잘리지 않음. 가상화된 목록이라 화면 밖 항목은 좌표가 (0,0,0,0) — 스크롤해서 렌더링시켜야 클릭 가능.

알림 카드의 "누적" 구조와 위치 기반 diff

같은 상대와 문자를 주고받을수록 알림 카드 하나가 "A" → "A B" → "A B C"처럼 줄이 계속 쌓입니다. 그래서 "마지막 줄만" 비교하는 대신, 발신자별로 지금까지 본 줄을 전부 기억해뒀다가 매 폴링마다 "카드 전체 줄 목록이 지난번 목록 뒤에 그대로 이어붙었는지"를 위치로 비교해 새 줄만 콜백에 넘깁니다(`_new_lines_since()`). 내용이 아니라 위치로 비교하기 때문에, 완전히 같은 문구를 짧은 간격으로 여러 번 보내도 매번 정확히 새 줄로 잡힙니다 — 내용 비교였다면 두 번째부터 "이미 본 문구"로 착각해 건너뛰었을 것입니다.

발신자가 "나"로 뜨는 카드 복구

우리가 마지막으로 답신을 보낸 뒤로는 상대가 새 문자를 보내기 전까지 카드의 발신자 표시가 계속 "나"로 남아있는 게 실제로 확인됐습니다. 예전 코드는 이 경우 항목을 통째로 버려서 그 번호의 진행 상태를 갱신하지 못하고 계속 건너뛰었는데, 다행히 진짜 상대방 번호가 본문 후보 줄들 사이에 소재목처럼 다시 나오는 걸 발견해 이를 복구에 씁니다 — 단, 방향성 격리 문자(U+2068~U+2069)로 감싸인 "필드성" 값일 때만 소재목으로 인정해서, 상대가 문자 내용으로 진짜 전화번호를 보낸 경우와 구분합니다.

bidi 문자 제거

발신자/시각 필드에 윈도우 UI가 자동으로 끼워넣는 양방향 텍스트 제어 문자(예: "`010-2405-3466`")가 `window_text()` 로 읽으면 그대로 섞여 나와, 정규식 매칭 전에 `_clean_bidi()` 로 제거해야 합니다.

발송: 기존 대화 찾기 → 새 메시지 폴백

대화 목록 행의 접근성 이름("{번호}와의 대화 메시지 미리 보기 ...")을 정리한 텍스트로 직접 비교해 매칭합니다(pywinauto의 `title_re` 는 bidi 문자 때문에 매칭 실패). 못 찾거나 스크롤해도 렌더링 안 되거나 클릭이 안 먹으면 "새 메시지"로 폴백 — 같은 번호를 입력해도 앱이 기존 대화로 이어주므로 안전망 역할을 합니다.

중복 방지 3중 구조

1. `dedup_key` (UNIQUE 제약, `INSERT OR IGNORE`) — 같은 화면을 반복 폴링해도 DB에 한 번만 저장
2. `self_echo` 확인(`api_save_message`) — 우리가 보낸 답신이 알림에 다시 잡혀 수신 문자로 오인되는 걸, 최근 1일 내 같은 번호·같은 문구의 발신 기록과 대조해 걸러냄
3. 재시작 시 `_load_recent_seen()` — `watch_daemon.py` 를 켜다 켜면 `phone_link`의 "이미 처리한 내용" 메모리가 초기화되는데, 서버가 이미 아는 내용을 재시작 전 불러와 `seen_lines_by_sender` 에 미리 채워 중복 저장을 방지

heartbeat

`watch_daemon.py` 가 매 폴링 주기(기본 5초)마다 `/api/heartbeat` 를 호출합니다.
`app.py` 는 마지막 시각을 `settings` 에 저장하고, `/api/status` 가 `WATCHER_STALE_SECONDS=60` 초 이상 신호가 없으면 `alive=False` 로 응답 — 대시보드 상단 "수신 감시 중단" 배너가 이 값을 씁니다. 신호를 한 번도 못 받은 상태(`last_seen` 없음)도 "정상"으로 단정하지 않고 같은 경고로 처리됩니다.

08 · 업무외 자동발송

`_maybe_send_auto_reply()` 하나에 이 기능의 모든 판단이 모여 있습니다.

업무시간은 게이트가 아니라 표시値

토글이 켜져 있으면 평일 업무시간(`_is_business_hours()`, 09:00–18:00)이 되어도 자동으로 꺼지거나 막히지 않습니다. 요일만으로는 공휴일을 판별할 수 없어서(평일이지만 쉬는 날), "업무시간이니 자동 대기"하는 판단을 시스템이 대신하지 않고 담당자가 토글로 직접 켜고 끄도록 설계를 뒤집은 것 — 화면(`/api/status`, `/api/settings/auto_reply` 둘 다 `business_hours_now` 를 내려줌)은 참고 정보로만 보여줍니다.

스팸 방지 (`quiet_minutes`)

같은 번호로 문자를 나눠 보내는 경우처럼 짧은 시간 안에 여러 통이 오면, 매 통마다 자동발송이 나가는 걸 막기 위해 최근 `quiet_minutes` (기본 1분, `settings` 에서 실시간 변경 가능)분 안에 이미 발신 기록이 있으면 건너뛵니다.

발송 전 기록 → 실패 시 롤백

DB 기록을 발송보다 먼저 남기고(`direction='out'`, `auto_sent=1`), `phone_link.send_message()` 가 예외를 던지면 그 행을 삭제하는 순서를 씁니다. 반대 순서(발송 후 기록)였다면 "문자는 실제로 나갔는데 서버 재시작(git pull로 인한

use_reloader 재시작 타이밍 등) 때문에 기록 코드까지 못 갔다"는, 원인 파악이 훨씬 어려운 실패 모드로 이어질 수 있습니다.

설정 키(SETTINGS 테이블)	기본값	설명
auto_reply_enabled	"0"	켜짐/꺼짐
auto_reply_template_id	(없음)	발송할 상용문구. 꺼져 있어도 값은 유지될 수 있음
auto_reply_quiet_minutes	"1"	스팸 방지 윈도우(분)

09 · 프론트엔드 구조

SPA 프레임워크 없이 `dashboard.html` (586줄, 인라인 CSS) + `dashboard.js` (1009줄, 바닐라 JS)로 이뤄진 단일 페이지입니다.

폴링 주기

함수	주기	이유
loadMessages	15초	워치가 새로 저장한 수신 문자를 화면에 자동 반영
loadStatus	10초	감시 중단/자동발송 경고를 좀 더 자주 확인
loadAutoReplySettings	10초	업무시간이 바뀌면 안내 문구도 따라가고, 다른 PC에서 켜고/꺼것도 빨리 반영

목록/자동발송 설정 폴링은 지금 그 입력 칸에 포커스가 있으면(타이핑 중이면) 값을 덮어쓰지 않도록 `document.activeElement` 를 확인합니다 — 10~15초마다 값이 갱신되는데 타이핑을 방해하면 안 되기 때문입니다.

커스텀 토글 스위치의 클릭 가로채기 문제

업무외 자동발송 토글은 `<input type=checkbox>` (`opacity:0`)와 그 위를 덮는 `.track <span>` 으로 구현되어 있습니다. 일반적인 `.click()` 은 실제 `input`이 아니라 위에 그려진 `.track` 이 클릭을 가로채 씹힐 수 있어, 자동화/테스트 코드에서는 `force=True` 클릭이 필요합니다(사람이 마우스로 누르는 것 자체는 정상 동작).

자동발송 저장 순서 의존성

`saveAutoReplySettings()` 는 `enabled` 가 `true`인데 `templateId` 가 비어 있으면 저장을 거부하고 체크박스를 도로 끕니다(`rename_image` 류의 "친절한 되돌림" 패턴과 동일한 취지). 그래서 화면·자동화 스크립트 모두 **문구 선택** → **토글 켜기** 순서를 지켜야 하며, 반대로 하면 토글이 켜지자마자 꺼지는 것처럼 보입니다.

10 · 운영 노트

기동 · 종료

```
start_server.bat
# → requirements.txt pip install(먹등) → logs\ 폴더 생성
#   → app.py, watch_daemon.py를 각각 창 없이 백그라운드로 기동
#   → 표준출력/에러를 logs\app_YYYY-MM-DD.log 등으로 리다이렉트

stop_server.bat
# → CommandLine에 app.py 또는 watch_daemon.py가 포함된 python.exe 프로세스만 골라 종료
#   (taskkill /IM python.exe처럼 전체를 죽이지 않음)
```

템플릿 자동 재로드

`app.config["TEMPLATES_AUTO_RELOAD"] = True` — git pull로 `dashboard.html` 이 바뀌어도 서버 재시작 없이 다음 요청부터 바로 반영됩니다(Flask 기본값은 템플릿을 한 번 읽으면 계속 캐시).

설치 의존성

`requirements.txt` 는 `pywinauto` / `pywin32` 를 `sys_platform == "win32"` 조건부로 선언합니다 — 리눅스/개발 환경에서 `pip install -r requirements.txt` 를 그대로 돌려도 이 둘은 건너뛰어지고, 대시보드 서버 자체(메시지 목록/상용문구 관리)는 문제없이 동작합니다.

데모 데이터

`seed_demo_data.py` 가 고정된 `DEMO_PHONES` 번호로 데모 민원/답신을 삽입합니다. `--clear` 로 그 데모 데이터만 안전하게 제거할 수 있어(실제 데이터는 건드리지 않음), 스크린샷·시연용 데이터를 채웠다 지우는 용도로 씁니다.

알려진 제약

- 한 PC에서 한 세트의 휴대폰과 연결 창만 조작 가능 — 여러 회선을 동시에 감시하려면 PC/창을 분리해야 함
- `app.run()` 이 `threaded=False` 라 요청이 완전히 직렬 처리됨 — 자동발송을 별도 스레드로 뺀 것도 이 제약을 우회하기 위함(02장)
- 발송은 창이 화면에 실제로 보이는 상태여야 동작 — 최소화된 채로는 불가능해, 발송 직전 자동 복원 후 다시 최소화

이 문서는 `app.py`, `db.py`, `phone_link.py`, `watch_daemon.py`, `templates/dashboard.html`, `static/js/dashboard.js`를 직접 읽고 작성되었습니다. 기능이 추가될 때마다 이 문서도 같이 갱신하는 것을 권장합니다.